

Contributing to OpenCV

Fixing Signed Left-Shift Undefined Behavior in Drawing Code

李语尚

Student ID: 12412308

CS219: Advanced Programming

Project 4

2026 年 5 月 30 日

目录

1	背景与任务选择	3
1.1	Project 4 简介	3
1.2	任务概述	3
2	问题分析	3
2.1	触发场景	3
2.2	根因分析	4
2.3	受影响的函数	4
3	修复方案	4
3.1	设计原则	4
3.2	新增 helper 函数	5
3.3	修改范围 (Before / After)	5
3.3.1	ThickLine()	5
3.3.2	FillConvexPoly()	6
3.3.3	CollectPolyEdges()	6
3.4	方案比较	7
3.5	回归测试	7
4	验证与测试	8
4.1	UBSan 构建与复现	8
4.2	修复前	8
4.3	修复后	9
5	性能评估	9
5.1	测试方法	9
5.2	正常正坐标数据 (offset = (0, 0))	10
5.3	负 offset 数据 (offset = (-120, -120))	10
5.4	性能结论	10
6	修改文件总结	11
7	困难与反思	11
7.1	遇到的困难	11
7.2	AI 工具使用	11
7.3	收获与体会	12
8	总结	13

1 背景与任务选择

1.1 Project 4 简介

本次 project 要求向开源项目 OpenCV 提交一个有意义的贡献。OpenCV 是世界上最广泛使用的计算机视觉库之一，主要使用 C++ 编写，被学术界和工业界广泛采用。

在 Project 4 的三个等级中 (Level A: 初学者、Level B: 中级、Level C: 高级)，我选择了 **Level C** —解决真实的 OpenCV Issue。具体而言，我修复了 OpenCV issue #28940：绘图模块中负坐标导致的有符号整数左移 undefined behavior。

1.2 任务概述

问题出在 imgproc 模块的绘图实现中。当用户调用 `cv::drawContours()` 并传入负的 `offset` 参数时，轮廓点可能被移动到图像边界外形成负坐标。负坐标本身是合法输入（表示图形部分位于图像外，需要裁剪），但原始代码在 fixed-point 坐标转换时对负的有符号整数执行左移，触发了 C++ undefined behavior。

- **Issue:** #28940
- **涉及文件:** `modules/imgproc/src/drawing.cpp`
- **修复方式:** 将 `signed left shift` 替换为乘法缩放
- **影响范围:** `ThickLine()`, `FillConvexPoly()`, `CollectPolyEdges()`

2 问题分析

2.1 触发场景

一个现实场景是：用户从原图中裁剪出 ROI，然后希望把原图坐标系中的轮廓绘制到 ROI 图像中。例如：

```
cv::Mat roi(80, 80, CV_8UC3, cv::Scalar::all(0));

std::vector<std::vector<cv::Point>> contours = {{
    cv::Point(100, 100), cv::Point(160, 100),
    cv::Point(160, 160), cv::Point(100, 160)
}};

cv::Point offset(-120, -120);

cv::drawContours(roi, contours, -1, cv::Scalar(0, 255, 0),
```

```
2, cv::LINE_8, cv::noArray(), INT_MAX, offset);
```

第一个轮廓点 (100, 100) 加上 offset (-120, -120) 后变成了 (-20, -20)。这是合法的输入——OpenCV 应该裁剪后绘制可见部分。

2.2 根因分析

drawing.cpp 中定义了 fixed-point 坐标缩放常量：

```
enum { XY_SHIFT = 16, XY_ONE = 1 << XY_SHIFT };
```

绘图模块内部使用 16.16 定点数格式，因此会将普通坐标放大 2^{16} 倍。原始代码在 ThickLine() 中使用了 signed 左移：

```
p0.x <<= XY_SHIFT - shift;  
p0.y <<= XY_SHIFT - shift;
```

当 $p0.x == -20$ 时，等价于执行 $-20 \ll 16$ 。根据 C++ 标准，对负的 signed integer 进行左移是 **undefined behavior**。普通 Release 构建下，x86 平台使用补码表示，可能看起来可以运行；但 C++ 标准不保证这种行为，且 UBSan 会在运行时检测并终止程序。

此外，粗线绘制会将线段扩展成四边形后调用 FillConvexPoly() 填充——第一次只修复 ThickLine() 后，同一输入会继续触发 FillConvexPoly() 中的同类问题。因此需要修复同一绘图路径上的所有 fixed-point 转换。

2.3 受影响的函数

以下三个函数均可能接收由负 offset 产生的负坐标：

1. ThickLine() ——粗线绘制，约 line 1669–1672
2. FillConvexPoly() ——凸多边形填充，约 line 1128–1150
3. CollectPolyEdges() ——多边形边收集，约 line 1274–1321

3 修复方案

3.1 设计原则

1. 禁止负坐标——负坐标是合法输入，表示图形部分位于图像外。
2. 不 clamp 到 0 ——这会导致图形形状变形，绘制结果不正确。
3. 保持原 fixed-point 缩放语义—— $x \ll 16 \approx x \times 65536$ 。

4. 避免 signed negative left shift UB ——使用乘法替代。

3.2 新增 helper 函数

```
// Use multiplication to avoid undefined behavior from
// left-shifting negative coordinates.
static inline int64
ScaleToFixedPoint( int64 v, int shift )
{
    CV_Assert( 0 <= shift && shift <= XY_SHIFT );
    const int64 factor = static_cast<int64>(1) << (XY_SHIFT - shift)
        ;
    CV_DbgAssert( v >= std::numeric_limits<int64>::min() / factor &&
        v <= std::numeric_limits<int64>::max() / factor );
    return v * factor;
}
```

关键设计要点：

- factor 是 $2^{XY_SHIFT - shift}$ ，始终为非负整数。左移非负数在 C++ 中是定义良好的。
- 返回值使用乘法：v * factor。对于非负数，结果与左移一致；对于负数，乘法是 C++ 定义良好的行为。
- CV_DbgAssert 在 Debug 模式下检测溢出，保证安全。

3.3 修改范围 (Before / After)

3.3.1 ThickLine()

Before:

```
p0.x <<= XY_SHIFT - shift;
p0.y <<= XY_SHIFT - shift;
p1.x <<= XY_SHIFT - shift;
p1.y <<= XY_SHIFT - shift;
```

After:

```
p0.x = ScaleToFixedPoint(p0.x, shift);
p0.y = ScaleToFixedPoint(p0.y, shift);
p1.x = ScaleToFixedPoint(p1.x, shift);
p1.y = ScaleToFixedPoint(p1.y, shift);
```

3.3.2 FillConvexPoly()

Before:

```
p0.x <=< XY_SHIFT - shift;
p0.y <=< XY_SHIFT - shift;
// ...
p.x <=< XY_SHIFT - shift;
p.y <=< XY_SHIFT - shift;
// ...
xs <=< XY_SHIFT - shift;
xe <=< XY_SHIFT - shift;
```

After:

```
p0.x = ScaleToFixedPoint(p0.x, shift);
p0.y = ScaleToFixedPoint(p0.y, shift);
// ...
p.x = ScaleToFixedPoint(p.x, shift);
p.y = ScaleToFixedPoint(p.y, shift);
// ...
xs = ScaleToFixedPoint(xs, shift);
xe = ScaleToFixedPoint(xe, shift);
```

3.3.3 CollectPolyEdges()

Before:

```
pt0.x = (pt0.x + offset.x) << (XY_SHIFT - shift);
// ...
pt1.x = (pt1.x + offset.x) << (XY_SHIFT - shift);
// ...
pt0c.x = (int64)(t0.x) << XY_SHIFT;
pt1c.x = (int64)(t1.x) << XY_SHIFT;
// ...
t0.y = pt0.y << XY_SHIFT;
t1.y = pt1.y << XY_SHIFT;
```

After:

```
pt0.x = ScaleToFixedPoint(pt0.x + offset.x, shift);
// ...
pt1.x = ScaleToFixedPoint(pt1.x + offset.x, shift);
// ...
```

```
pt0c.x = ScaleToFixedPoint(t0.x, 0);
pt1c.x = ScaleToFixedPoint(t1.x, 0);
// ...
t0.y = ScaleToFixedPoint(pt0.y, 0);
t1.y = ScaleToFixedPoint(pt1.y, 0);
```

3.4 方案比较

我也考虑了其他修复方案：

方案 A: 使用乘法缩放 (已采用)

```
return v * (static_cast<int64>(1) << (XY_SHIFT - shift));
```

优点：语义清晰，对负数定义良好，易于解释，性能影响可忽略。

方案 B: 转为 unsigned 后左移

```
return static_cast<int64>(static_cast<uint64_t>(v) << shift);
```

优点：避免 signed left shift UB，与补码行为接近。

缺点：unsigned 转 signed 在极端值上容易引发维护者讨论，可读性不如乘法。

方案 C: 负坐标 clamp 到 0

不推荐。负坐标是合法输入，clamp 会改变几何形状导致绘制不正确。

综上，采用 **方案 A**。

3.5 回归测试

在 modules/imgproc/test/test_drawing.cpp 中新增回归测试：

```
TEST(Drawing, drawContours_negative_offset_regression_28940)
{
    Mat roi(80, 80, CV_8UC1, Scalar::all(0));
    vector<vector<Point>> contours(1);
    contours[0].push_back(Point(100, 100));
    contours[0].push_back(Point(160, 100));
    contours[0].push_back(Point(160, 160));
    contours[0].push_back(Point(100, 160));

    drawContours(roi, contours, -1, Scalar(255), 2,
                LINE_8, noArray(), INT_MAX, Point(-120, -120));

    // 轮廓移动到 (-20,-20) 到 (40,40)，只有右下部分可见
    EXPECT_NE(0, roi.at<uchar>(20, 40));
```

```

    EXPECT_NE(0, roi.at<uchar>(40, 20));
    EXPECT_NE(0, roi.at<uchar>(40, 40));
    EXPECT_EQ(0, countNonZero(roi(Rect(60, 60, 20, 20))));
}

```

该测试覆盖了：

1. **可见区域：** (20,40), (40,20), (40,40) 应该有像素被绘制——验证裁剪后的轮廓仍然正确落在 ROI 内。
2. **越界区域：** ROI 右下角 (60,60) 到 (79,79) 应该全为 0——验证没有越界写入。

4 验证与测试

4.1 UBSan 构建与复现

为了在修复前确认 UB 的存在并在修复后验证它已被消除，我在 WSL 中构建了一个 UBSan 版本的 OpenCV，仅包含 core 和 imgproc 模块：

```

cmake -S . -B build_ubsan_28940 \
  -DCMAKE_BUILD_TYPE=Debug \
  -DBUILD_LIST=core,imgproc \
  -DCMAKE_CXX_FLAGS="-fsanitize=undefined_\
  \_\_\_\_fno-sanitize-recover=undefined_\
  \_\_\_\_fno-omit-frame-pointer"

cmake --build build_ubsan_28940 --target opencv_imgproc -j8

```

使用复现程序 test_28940.cpp 调用 drawContours()，分别加载原始库和修复后库。

4.2 修复前

```

$ LD_PRELOAD=./build_ubsan_28940/lib/libopencv_imgproc.so \
  ./test_28940

Calling drawContours with offset [-120, -120].
The first point becomes [-20, -20]

drawing.cpp:1662:10:
runtime error: left shift of negative value -20

```



```
#0 ThickLine drawing.cpp:1662
#1 cv::drawContours drawing.cpp:2571
#2 main test_28940.cpp:41
```

UBSan 明确检测到 left shift of negative value, 证实了 undefined behavior 的存在。

4.3 修复后

```
$ LD_PRELOAD=./build_ubsan_fixed/lib/libopencv_imgproc.so \
./test_28940

Calling drawContours with offset [-120, -120].
The first point becomes [-20, -20]

drawContours finished, pixel sum = [0, 61710, 0, 0]
```

修复后:

1. drawContours() 正常完成绘制。
2. UBSan 不再报告错误。
3. 输出图像非空——像素和 = [0, 61710, 0, 0], 验证图像确实被绘制了。

5 性能评估

5.1 测试方法

为了确认修复不会明显影响正常使用性能, 我分别使用原始 Release 库和修复后 Release 库运行 benchmark。测试配置如下:

- 构建类型: Release
- 不启用 UBSan
- 模块: core, imgproc
- 每组重复 5 次, 记录 best 和 avg
- 输出 checksum 保持一致以确认绘制结果相同

表 1: 正常坐标下 best time 对比

迭代次数	原始 best (ms)	修复 best (ms)	差异
100	32.56	31.84	-2.21%
1000	327.38	321.65	-1.75%
10000	3228.03	3252.81	+0.77%

表 2: 正常坐标下 avg time 对比

迭代次数	原始 avg (ms)	修复 avg (ms)	差异
100	32.89	32.29	-1.84%
1000	328.55	326.96	-0.48%
10000	3274.25	3268.45	-0.18%

5.2 正常正坐标数据 (offset = (0, 0))

输出 checksum 一致: 1.19493×10^7 。

5.3 负 offset 数据 (offset = (-120, -120))

表 3: 负 offset 下 best time 对比

迭代次数	原始 best (ms)	修复 best (ms)	差异
100	30.33	30.79	+1.51%
1000	304.72	305.12	+0.13%
10000	2997.30	3033.59	+1.21%

输出 checksum 一致: 1.07391×10^7 。

5.4 性能结论

修复后的性能与原始版本基本一致, 差异大多在 0%-2% 范围内, 属于正常运行噪声。原因有三:

1. 修改只发生在少量几何点的 fixed-point 坐标转换处。
2. 绘图函数主要耗时在裁剪、扫描线填充和像素写入。
3. $v * (1 \ll k)$ 是乘以 2 的幂, Release 编译器通常会优化为等价的低成本指令 (LEA 或位移 + 加法)。

因此, 本修复基本不影响性能。

表 4: 负 offset 下 avg time 对比

迭代次数	原始 avg (ms)	修复 avg (ms)	差异
100	30.67	31.34	+2.19%
1000	307.94	309.08	+0.37%
10000	3023.27	3056.36	+1.09%

6 修改文件总结

表 5: 修改文件清单

文件	修改性质
modules/imgproc/src/drawing.cpp	新增 ScaleToFixedPoint 函数 + 替换 17 处 left shift
modules/imgproc/test/test_drawing.cpp	新增回归测试

7 困难与反思

7.1 遇到的困难

1. **UBSan 一次报错不全**: 最初只修复 ThickLine() 后, 以为已经解决了问题。但重新运行测试后发现粗线绘制内部调用的 FillConvexPoly() 也有同样的 signed left shift。这说明在大型代码库中, 同一类 bug 经常存在于多个位置, 需要系统性地排查整个调用链。
2. **Replace 模式的保守性**: OpenCV 是一个非常成熟的项目, 任何对核心函数的修改都必须谨慎。我最初的修改方案在 CV_DbgAssert 中使用了除法来检测溢出, 需要确保除法不会引入新的性能问题 (好在 Debug 模式不追求极致性能)。
3. **回归测试的边界选择**: 写一个既简洁又能覆盖关键路径的测试并不容易。需要确保测试点落在可见区域和不可见区域的边界上, 且不受具体绘制实现细节 (如像素级偏移) 的影响。

7.2 AI 工具使用

在本项目中, 我使用了 CodeWhale (DeepSeek V4) 作为 AI 辅助编程工具。具体用途包括:

- **代码库分析**: 阅读和搜索 OpenCV 的 `drawing.cpp` (约 2900 行), 定位所有 signed left shift 的位置。CodeWhale 的并行搜索能力在这个阶段非常高效——它能同时 grep 多个符号并交叉验证。
- **修复方案推演**: 讨论了「乘法」vs「unsigned cast」两种方案, 选择更易维护和解释的乘法方案。
- **Benchmark 代码编写**: 生成了 `bench_28940.cpp` 和 `bench_28940_negative.cpp`, 用于系统的 before/after 性能对比。
- **测试代码编写**: 帮助设计了回归测试的验证逻辑和 assert 条件。
- **报告撰写**: 协助整理和结构化本报告的初稿。

与之前 project 中 AI 主要是“写代码”不同, 这次 AI 更像是“结对编程的资深工程师”——帮助理解大型项目的代码结构、设计方案、验证思路。真正的决策(选择什么方案、如何测试、提交什么内容)仍然由我自己做出。

7.3 收获与体会

这次 project 最大的收获不是学会了某个具体技术, 而是体验了一次完整的 **真实开源贡献流程**:

1. **从零构建一个大型项目**: OpenCV 的 CMake 构建系统非常复杂, 需要理解模块依赖、编译选项、sanitizer 集成等。与之前的 homework 项目不同, 这不是从头写一个小程序, 而是在一个千万行级别的代码库中做手术式的修改。
2. **阅读和理解他人代码**: `drawing.cpp` 中包含了 Bresenham 画线、多边形扫描线填充、抗锯齿等图形学算法。虽然不需要完全理解每一步, 但需要理解数据流——坐标如何从外部传入、经过哪些转换、最终如何写入图像。
3. **C++ undefined behavior 的可怕之处**: Release 模式下 `-20 << 16` 在 x86 上恰好“工作正常”, 但这是被标准明确定义为 UB 的操作。编译器完全有权生成不正确的代码。UBSan 是这类问题的照妖镜——没有它, 这类 bug 可能会潜伏数年。
4. **开源社区的工作方式**: 从 issue 报告到 PR 提交, 每一步都需要清晰的沟通、可复现的证据、对审核者的尊重。修复代码是一部分, 写好 commit message 和 PR 描述是同样重要的一部分。
5. **性能验证的重要性**: 在像 OpenCV 这样的性能敏感项目中, 即使是修复 bug 也需要提供性能数据证明没有退化。这迫使我建立了系统的 benchmark 流程。

Project-based learning 确实是成为真正程序员的最佳途径。这次 project 让我从“写代码的人”向“软件工程师”迈进了一步。

8 总结

本次 project 修复了 OpenCV imgproc 绘图模块在负坐标场景下的 C++ undefined behavior。修复将 signed left shift 替换为乘法缩放，在保持语义不变的前提下消除了 UB。

修复前：

```
drawContours(..., offset=(-120,-120))  
→ ThickLine() → p0.x <= 16  
→ left shift of negative value  
→ UBSan 报错，程序终止
```

修复后：

```
drawContours(..., offset=(-120,-120))  
→ fixed-point 坐标转换使用乘法缩放  
→ 无 UBSan 错误  
→ 图像正常绘制，checksum 一致
```

该修复保持绘制结果与原始版本一致，性能影响在 0%-2% 范围内可以忽略，适合作为一个独立的、真实的、可测试的 OpenCV bug fix PR。

GitHub 仓库

- Fork: <https://github.com/lys0204/opencv>
- Branch: project4_qdB3ar
- Commit: 434bb82a8878 — imgproc: avoid signed left-shift UB in drawing code

参考文献

- [1] OpenCV Issue #28940: *[UB] Left Shift of Negative Value in cv::ThickLine When Drawing Contours with Negative Offset.*
<https://github.com/opencv/opencv/issues/28940>
- [2] OpenCV Contribution Guide.
https://github.com/opencv/opencv/wiki/How_to_contribute
- [3] ISO/IEC 14882:2020 — Programming Language C++.
Section 7.6.7: Shift operators. Left shift of a negative signed integer is undefined behavior.

- [4] LLVM Documentation: UndefinedBehaviorSanitizer.
<https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>
- [5] OpenCV GitHub Repository.
<https://github.com/opencv/opencv>
- [6] My OpenCV Fork.
<https://github.com/lys0204/opencv>